

Technical Specification and Dataset Architecture for Training AI Code Reviewers: Java Best Practices, Anti-Patterns, and Fine-Tuning Benchmarks

Architectural Foundations and Creational Mechanics

To develop a production-grade artificial intelligence code reviewer, a training dataset must be grounded in the structural mechanics of object lifecycle management, memory allocators, and JVM runtime operations.³ Simple rule-based or regex static analysis tools identify superficial syntax anomalies, but they are blind to the semantic design decisions that define secure, high-performance, and maintainable software.⁷ An AI code reviewer must be trained on paired, tokenized sequences of non-compliant and refactored Java code to learn the semantic representations of creational design.¹⁰

Static Factory Methods and Naming Conventions

The traditional reliance on public constructors restricts design flexibility, enforces concrete subclass binding, and prevents object reuse.³ Static factory methods address these issues by providing named abstractions that decouple instance creation from the concrete implementation class.³

Java

// Non-Compliant: Redundant object allocation via constructor

```
String id = new String("custom-unique-identifier")
```

// Compliant: Static factory utilizing pre-compiled caching mechanics

```
String optimizedId = String.valueOf("custom-unique-identifier")
```

Static factory methods do not require a new object allocation on every invocation, enabling the reuse of pre-constructed, cached instances.³ This capability reduces the garbage collection overhead for highly populated immutable structures.³ To build an effective dataset for training

an AI reviewer, instances must be labeled with standard creational naming conventions that indicate the exact type-conversion and instance-caching semantics.³

Factory Method Prefix	Type of Operation	Lifecycle Guarantee	Architectural Example
from ³	Single-parameter type conversion ³	Creates or retrieves a translated instance ³	Date.from(instant) ³
of ³	Multi-parameter aggregation ³	Combines parameters into a unified collection ³	EnumSet.of(JACK, QUEEN, KING) ³
valueOf ³	Verbose type translation ³	Typically returns cached, highly reused values ³	BigInteger.valueOf(Integer.MAX_VALUE) ³
instance / getInstance ³	Context-specific parameter matching ³	May return cached singletons or shared instances ³	StackWalker.getInstance(options) ³
create / newInstance ³	Explicit structural initialization ³	Guarantees the return of a distinct, new instance ³	Array.newInstance(classObject, length) ³
getType ³	Cross-class retrieval ³	Returns an instance located in a distinct class ³	Files.getStore(path) ³
newType ³	Cross-class instantiation ³	Instantiates a new type located in a distinct class ³	Files.newBufferedReader(path) ³
type ³	Concise factory translation ³	Alternative to more verbose cross-class factory helpers ³	Collections.list(legacyLitany) ³

Constructor Overload Proliferation and the Builder Pattern

As enterprise classes scale, constructors often accumulate optional parameters, leading to the telescoping constructor pattern.³ This design forces clients to pass arbitrary placeholder values

(such as 0 or null) to satisfy constructor signatures, creating a high cognitive load and making code hard to read.³

The JavaBeans pattern—using parameterless constructors followed by sequential setter calls—solves telescoping signatures but introduces a severe thread-safety issue.³ It splits class initialization across multiple lines of code, leaving objects in an inconsistent, partially constructed state during thread context switches, and prevents the creation of immutable classes.³ The Builder pattern combines the safety of telescoping constructors with the readability of JavaBeans, allowing stepwise parameters to build an immutable class.⁷

```
Java
// Non-Compliant: Telescoping constructor pattern is fragile and unreadable
public class ConnectionConfig
    private final String hostname
    private final int port
    private final int timeout
    private final boolean ssl

    public ConnectionConfig(String hostname)
        this.hostname = hostname
        this.port = 80
    }

    public ConnectionConfig(String hostname, int port)
        this.hostname = hostname
        this.port = port
    }

    public ConnectionConfig(String hostname, int port, int timeout)
        this.hostname = hostname
        this.port = port
        this.timeout = timeout
    }

    public ConnectionConfig(String hostname, int port, int timeout, boolean ssl)
        this.hostname = hostname
        this.port = port
        this.timeout = timeout
        this.ssl = ssl
    }
}
```

Java

// Compliant: Builder pattern preserves immutability and readability

```
public class ConnectionConfig
```

```
    private final String hostname
```

```
    private final int port
```

```
    private final int timeout
```

```
    private final boolean ssl
```

```
    private ConnectionConfig(Builder builder)
```

```
    { this.hostname = builder.hostname
```

```
      this.port = builder.port
```

```
      this.timeout = builder.timeout
```

```
      this.ssl = builder.ssl
```

```
    }
```

```
    public static class Builder
```

```
    { private final String hostname
```

```
      private int port = 80
```

```
      private int timeout = 5000
```

```
      private boolean ssl = false
```

```
      public Builder(String hostname)
```

```
      { this.hostname = hostname
```

```
      }
```

```
      public Builder port(int val)
```

```
      { port = val; return this
```

```
      }
```

```
      public Builder timeout(int val)
```

```
      { timeout = val; return this
```

```
      }
```

```
      public Builder ssl(boolean val)
```

```
      { ssl = val; return this
```

```
      }
```

```
      public ConnectionConfig build()
```

```
      { return new ConnectionConfig(this
```

```
      }
```

```
    }
```

```
}
```

Immutability Standards and Instantiability Controls

To enforce true class immutability, developers must block all mutation paths and subclass overrides.³ This requires declaring the class as final, making all fields private and final, and ensuring exclusive, defensively copied access to any mutable internal components.³

```
Java
// Compliant: Fully immutable class with defensive copy implementation
public final class ImmutablePeriod
{
    private final java.util.Date start;
    private final java.util.Date end;

    public ImmutablePeriod(java.util.Date start, java.util.Date end)
    {
        this.start = new java.util.Date(start.getTime()); // Defensive copy
        this.end = new java.util.Date(end.getTime()); // Defensive copy
        if (this.start.compareTo(this.end) > 0)
        {
            throw new IllegalArgumentException(this.start + "after" + this.end);
        }
    }

    public java.util.Date start()
    {
        return new java.util.Date(start.getTime()); // Defensive copy on retrieval
    }

    public java.util.Date end()
    {
        return new java.util.Date(end.getTime()); // Defensive copy on retrieval
    }
}
```

Additionally, utility classes designed only to group static helper methods must not be instantiable.³ Developers sometimes mistakenly define these utility classes as abstract to prevent instantiation.³ However, this is an anti-pattern: the class can still be subclassed, and those subclasses can be instantiated.³ The correct approach is to enforce non-instantiability using a private constructor that explicitly throws an exception.³

Java

// Non-Compliant: Abstract modifier does not prevent subclass instantiation

```
public abstract class ArrayUtils
```

```
    public static void swap(int arr, int i, int j)
```

```
    {
        int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
    }
}
```

Java

// Compliant: Private constructor prevents instantiation and subclassing

```
public final class ArrayUtils
```

```
    private ArrayUtils()
```

```
    {
        throw new AssertionError("Instantiation of utility class is prohibited.");
    }

```

```
    public static void swap(int arr, int i, int j)
```

```
    {
        int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
    }
}
```

Another critical error is using primitive floating-point types (float and double) for exact financial and mathematical calculations.⁷ Due to the binary representation of IEEE 754 floating-point standards, arithmetic operations on floats introduce rounding errors.⁷ Training datasets must teach AI reviewers to replace floating-point operations in financial code with BigDecimal or scaled integer types.⁷

Java

// Non-Compliant: Rounding errors in float-based financial calculations

```
double price = 1.00
double discount = 0.90
double result = price - discount // result evaluates to 0.09999999999999998

// Compliant: Exact arithmetic utilizing BigDecimal
java.math.BigDecimal precisePrice = new java.math.BigDecimal("1.00")
java.math.BigDecimal preciseDiscount = new java.math.BigDecimal("0.90")
java.math.BigDecimal preciseResult = precisePrice.subtract(preciseDiscount) // evaluates
to 0.10
```

Syntactic Code Smells, Naming Rules, and Cognitive Complexity

Code quality directly impacts readability and long-term maintainability.⁷ An AI reviewer must evaluate the cognitive load of control flow structures, ensuring that nested, hard-to-follow logic is refactored into clear, self-documenting code paths.¹²

Control Flow Nesting and Early Exception Interception

Deep nesting of conditional blocks creates "staircase code," which severely taxes a developer's working memory.¹² Each nested level represents a conditional constraint that must be kept in mind to understand the execution path.¹² Training datasets should teach AI reviewers to use a fail-fast pattern: validating arguments immediately at the method boundary, and returning or throwing exceptions early to flatten the control flow.³

```
Java
// Non-Compliant: Highly nested control flow (Staircase Pattern)
public double calculatePremium(User user, Policy policy)
    double premium = 0.0
    if (user != null
        if (user.isVerified())
            if (policy != null
                if (policy.isActive())
                    premium = policy.getBaseRate() * user.getRiskMultiplier()
            }
        }
    }
}
```

```
return premium;
```

Java

```
// Compliant: Fail-fast validation with flattened execution paths
```

```
public double calculatePremium(User user, Policy policy)
```

```
if (user == null || !user.isVerified() || policy == null || !policy.isActive())
```

```
return 0.0; // Early exit flattens the execution path
```

```
return policy.getBaseRate() * user.getRiskMultiplier();
```

Deconstruction of Satirical Agile Anti-Patterns

A common issue in agile environments is the misuse of refactoring principles in the name of speed.¹³ Some developers systematically erode clean, type-safe structures by eliminating useful generics, discarding final keywords, and replacing robust types with raw parameters.¹³ All reviewers must be trained to recognize this anti-pattern progression and understand how these changes weaken compile-time safety and increase the risk of runtime errors.¹³

Java

```
// Non-Compliant: Satirical refactoring steps that erode type safety
```

```
public class DBWrapper
```

```
// Erosion Step 1: Converting structured types to base class Object
```

```
// Erosion Step 2: Removing final modifiers and concrete generics
```

```
public static java.util.List search(Object query, Object database)
```

```
// Erosion Step 3: Hardwiring dependencies into static helper parameters
```

```
return ((Database) database).findAll((String) query); // Unsafe casting
```



```

Java
// Compliant: Strongly typed, object-oriented constructor injection
public final class DBWrapper
    private final Database db;

    public DBWrapper(Database db)
        this.db = java.util.Objects.requireNonNull(db, "Database dependency cannot be null");

    public java.util.List<Product> search(final String name)
        final java.util.List<Product> products = db.findAll(name);
        return products == null ? java.util.Collections.emptyList() : products;

```

Style Guidelines, Naming Context, and Logging Overhead

Clean naming conventions act as built-in documentation, reducing the need for redundant comments.¹² Poor naming—such as unclear single-character variables or inconsistent layouts—obscures the code's intent.² AI reviewer datasets should include standard style guides (such as the Google Java Style Guide) to identify and correct formatting issues, missing Javadocs, and raw type collections.²

Another common performance issue is string composition during logging.² Concatenating string variables inside logging statements constructs string objects in memory regardless of whether the target log level is active, wasting CPU cycles and memory.² The SLF4J placeholder pattern ({}) defers string compilation until after checking the active log level, preventing these unnecessary runtime allocations.²

```

Java
// Non-Compliant: High overhead from string concatenation in inactive log level
logger.debug("Failed transaction processing for cardholder: " + user.getName() + " with

```

```
transaction status: " + status.getCode());
```

```
// Compliant: Deferred string compilation utilizing SLF4J parameterized syntax
```

```
logger.debug("Failed transaction processing for cardholder: {} with transaction status: {}"
```

```
ser.getName(), status.getCode());
```

Optional Type Mechanics and Null-Safety Paradigms

Introduced in Java 8, Optional is a container object designed specifically to represent the potential absence of method return values. However, misuse of Optionals can introduce heap overhead, classloader leaks, and verbose code styles.

Optional Field and Parameter Anti-Patterns

A critical anti-pattern is utilizing Optional inside class fields or method parameters. This pattern degrades class layout efficiency and creates serialization and deserialization failures within standard frameworks (e.g., Jackson, JPA, Hibernate) that expect raw primitives or object structures rather than containers.

Java

```
// Non-Compliant: Optional as a class field causes memory overhead and serialization issues
```

```
public class UserProfile
```

```
    private String username;
```

```
    private Optional<String> email; // Severe anti-pattern: field encapsulation of Optional
```

```
    public void setEmail(Optional<String> email) // Anti-pattern: method parameter Optional
```

```
    {
        this.email = email;
    }
}
```

Java

```
// Compliant: Encapsulate raw nullable fields, and expose Optional selectively via getters
```

```
public class UserProfile
```

```

private String username
private String email; // Permitted internally to remain null

public void setEmail(String email)
    this.email = email;

public java.util.Optional<String> getEmail()
    return java.util.Optional.ofNullable(email); // Explicitly signals possible absence to callers

```

Imperative Interception vs. Functional Chaining

Many developers continue to use Optionals in an imperative style by matching `isPresent()` checking followed by a `.get()` call. This mimics traditional null checks and can cause a `NoSuchElementException` if modified without synchronization.

```

Java
// Non-Compliant: Verbose isPresent() + get() pattern mimics raw null checks
public String fetchUserPostCode(Optional<User> userOpt)
    if (userOpt.isPresent())
        User user = userOpt.get(); // Vuln: throws exception if user is missing
        Address address = user.getAddress();
        if (address != null)
            return address.getZipCode();
        return "UNKNOWN"

```

Java

```
// Compliant: Functional pipeline leveraging map, flatMap, and orElse
public String fetchUserPostCode(java.util.Optional<User> userOpt)
    return userOpt
        .map(User::getAddress)
        .map(Address::getZipCode)
        .orElse("UNKNOWN") // Completely null-safe functional traversal
```

Lazy vs. Eager Default Value Evaluation

Choosing between `orElse()` and `orElseGet()` has direct performance implications. The `orElse()` method evaluates its arguments *eagerly*, which means the default value is initialized even if the `Optional` is not empty. For expensive database transactions or network requests, this can waste substantial resources.

```
Java
// Non-Compliant: orElse eagerly instantiates fallback even if value exists
User defaultUser = userOpt.orElse(loadDefaultUserFromDatabase())

// Compliant: orElseGet uses a lazy Supplier, executing only when necessary
User defaultUser = userOpt.orElseGet(() -> loadDefaultUserFromDatabase())
```

JVM Memory Management, Resource Leakage, and Lifecycle State

An effective AI code reviewer must look beyond syntax correctness to analyze runtime safety.¹⁶ It should identify patterns that lead to resource depletion, such as memory leaks in custom state managers or leaked connections in persistent threads.

Obsolete References and Garbage Collection Scope

In the JVM, an object remains in memory as long as it is reachable from any live thread GC roots.⁶ When developers implement custom memory managers (such as arrays, buffers, or internal caches), they must manually manage object lifecycles.³ Under these conditions, the garbage collector cannot determine when an element is no longer needed.³ To prevent memory leaks, references to unused elements must be explicitly cleared.³

Java

// Non-Compliant: Memory leak due to reference retention in custom container

```
public class ArrayQueue
```

```
    private Object queue = new Object
```

```
    private int head = 0
```

```
    private int tail = 0
```

```
    public void enqueue(Object x) { queue[tail++] = x; }
```

```
    public Object dequeue()
```

```
    { if (head == tail) throw new java.util.NoSuchElementException(); }
```

```
    return queue[head++]; // Element remains referenced in the array, causing a memory leak
```

```
    }  
}
```

Java

// Compliant: Nulling obsolete reference to allow garbage collection

```
public class ArrayQueue
```

```
    private Object queue = new Object
```

```
    private int head = 0
```

```
    private int tail = 0
```

```
    public void enqueue(Object x) { queue[tail++] = x; }
```

```
    public Object dequeue()
```

```
    { if (head == tail) throw new java.util.NoSuchElementException(); }
```

```
    Object val = queue[head];
```

```
    queue[head] = null; // Clears the obsolete reference to let GC reclaim the memory
```

```
    head++;
```

```
    return val;
```

```
    }  
}
```

While clearing references manually is occasionally necessary, it should remain the exception.³

The best way to eliminate obsolete references is to design narrow scopes that allow variables to naturally fall out of scope when no longer needed.³

Performance Dynamics of String Manipulation in Loops

Strings in Java are immutable.¹¹ Appending strings inside a loop using the + operator forces the JVM to continuously create intermediate StringBuilder instances, copy the character arrays, and allocate new strings.¹¹ Traditional JIT compiler optimizations cannot optimize these allocations away, leading to severe memory churn.¹¹

To avoid this overhead, developers should initialize a StringBuilder with an estimated initial capacity.¹¹ This prevents unnecessary internal array resizing and copy actions.¹¹ Additionally, StringBuilder is preferred over StringBuffer for local variables because it avoids the overhead of synchronization.¹¹

```
Java
// Non-Compliant: Loop concatenation allocations create massive garbage collection pressure
String output = ""
for (Person p : persons)
    output += " " + p.getName() // Allocates a new String and StringBuilder on each iteration

```

```
Java
// Compliant: StringBuilder initialized with estimated buffer capacity
StringBuilder builder = new StringBuilder(persons.size() * 16 // Pre-allocates buffer size
for (Person p : persons)
    if (builder.length() > 0)
        builder.append(" ")
    builder.append(p.getName()) // Modifies the internal array in-place
String output = builder.toString()

```

Dynamic Action	Underlying JVM Mechanism	Memory Footprint Impact	Optimal Refactored Strategy
String Concatenation in Loops ¹¹	Compiles to sequential new StringBuilder().append().toString() loops ¹¹	High garbage collection pressure due to temporary object churn ¹¹	Use a pre-sized, single-instance StringBuilder ¹¹
Autoboxing Operations ³	Implicitly translates primitives to objects via .valueOf() ³	Generates large volumes of unnecessary wrapper objects on the heap ³	Use primitive types (long, double) directly in performance loops ³
String Equality Evaluation ¹¹	Checks memory references rather than matching characters ¹¹	Can evaluate to false for identical string values ¹¹	Use .equals() or null-safe constants like "value".equals(variable) ¹¹
Number to String Conversion ¹¹	"" + number compiles to StringBuilder operations ¹¹	Allocates intermediate StringBuilder wrappers unnecessarily ¹¹	Use explicit static converters like Integer.toString(value) ¹¹

ThreadLocal Lifecycle Hazards and ClassLoader Leaks

ThreadLocal variables associate values with the execution thread.¹⁷ However, in modern application servers like Apache Tomcat, worker threads are managed in persistent pools to handle HTTP requests.¹⁷ These threads never terminate; instead, they return to the pool to be reused for subsequent requests.¹⁷

If a ThreadLocal value is not manually cleared using the .remove() method before the thread is returned, the reference persists.¹⁷ If the object stored in the ThreadLocal was loaded by a custom web application class loader, this reference prevents the entire class loader from being garbage collected when the application is redeployed.⁴ This mechanism causes a progressive Metaspace memory leak, eventually crashing the JVM with an OutOfMemoryError.⁶

Live GC Root → Pool Thread → ThreadLocalMap → Entry (Value) → Application Class

Java

// Non-Compliant: ThreadLocal variables are retained indefinitely inside thread pools

public class UserRequestContext

private static final ThreadLocal<UserContext> CONTEXT = new ThreadLocal<>()

public static void setContext(UserContext user)

CONTEXT.set(user);

public static UserContext getContext()

return CONTEXT.get();

}

Java

// Compliant: Absolute resource release via finally boundary blocks

public class UserRequestContext

private static final ThreadLocal<UserContext> CONTEXT = new ThreadLocal<>()

public static void executeInContext(UserContext user, Runnable task)

try

CONTEXT.set(user);

task.run();

finally

CONTEXT.remove(); // Clears reference to prevent ClassLoader leaks

}

Similar leaks can occur with database connections left open after processing, or logging diagnostic contexts (like MDC) that store state in persistent thread-locals.⁵ AI reviewer datasets must highlight code patterns that lack explicit resource cleanups within finally blocks, suggesting the use of try-with-resources to ensure reliable cleanup.⁶

Java

```
// Non-Compliant: Database connection remains open if execution fails
public void logDatabaseStatus(javax.sql.DataSource ds) throws java.sql.SQLException {
    java.sql.Connection conn = ds.getConnection();
    java.sql.Statement stmt = conn.createStatement();
    java.sql.ResultSet rs = stmt.executeQuery("SELECT STATUS FROM SYSTEM_INFO");
    if (rs.next()) {
        logger.info("System status: " + rs.getString(1));
    }
    // Missed if an exception is thrown
}
```

Java

```
// Compliant: try-with-resources ensures resource cleanup
public void logDatabaseStatus(javax.sql.DataSource ds) throws java.sql.SQLException {
    try (java.sql.Connection conn = ds.getConnection();
        java.sql.Statement stmt = conn.createStatement();
        java.sql.ResultSet rs = stmt.executeQuery("SELECT STATUS FROM SYSTEM_INFO")) {
        if (rs.next()) {
            logger.info("System status: {}" + rs.getString(1));
        }
    }
    // Auto-closes all resources in reverse order, even if an exception occurs
}
```

Advanced Exception Handling and Control Flow

Integrity

Robust exception handling ensures that system failures are resolved safely.¹⁶ An AI reviewer must evaluate exception types, check logging configurations, and verify that failures leave the system in a consistent, predictable state.³

Checked versus Unchecked Exception Semantics

Checked exceptions are designed for conditions that an application should recover from, while unchecked exceptions (such as `NullPointerException` and `IllegalArgumentException`) indicate programmer or usage errors.³

Reusing standard JDK exceptions (like those in the table below) instead of creating custom types simplifies the API and makes it easier for other developers to learn.³ AI reviewer datasets must flag cases where custom exceptions are used when a standard exception is more appropriate, or where general types like `Throwable` or `Exception` are caught unnecessarily.³

Standard Exception Type	Primary Architectural Occasion for Use	Remediation Rule
<code>IllegalArgumentException</code> ³	A passed parameter is inappropriate for the method ³	Reject invalid input values at the method boundary ³
<code>IllegalStateException</code> ³	The object state is inappropriate for the invocation ³	Validate internal states before initiating modifications ³
<code>NullPointerException</code> ³	A parameter value is null where prohibited ³	Enforce validations with <code>Objects.requireNonNull()</code> ³
<code>IndexOutOfBoundsException</code> ³	An index parameter value is out of bounds ³	Validate indices against array or collection boundaries ³
<code>ConcurrentModificationException</code> ³	Unauthorized concurrent modification is detected ³	Use thread-safe structures or synchronize access paths ³
<code>UnsupportedOperationException</code> ³	The object does not support the method ³	Use for interface methods that the class does not implement ³

Exception Translation, Chaining, and Documentation

When an exception crosses abstraction layers, letting lower-level implementation details leak out degrades encapsulation.³ Higher layers should catch these lower-level exceptions and throw exceptions that are explained in terms of the higher-level abstraction.³ This is known as Exception Translation.³

To preserve the underlying root cause for debugging, developers should pass the original exception to the constructor of the new exception.³ This is known as Exception Chaining.³ Additionally, all thrown exceptions should be clearly documented in the Javadoc using the @throws tag.³

```
Java
// Non-Compliant: Lower-level SQL exception leaks from a higher-level user service
public User fetchUser(String id) throws java.sql.SQLException
    return database.queryUser(id) // Leaks implementation details of the persistence layer
```

```
Java
// Compliant: Exception translation with chained stack traces
/**
 * Retrieves the specified user record.
 *
 * @param id The unique identifier of the user.
 * @return The associated User object.
 * @throws UserNotFoundException If the requested user cannot be found in the database.
 */
public User fetchUser(String id)
    try
        return database.queryUser(id)
    catch (java.sql.SQLException e)
        throw new UserNotFoundException("Unable to retrieve user record for identifier: " + id, e) //
Exception Chaining
```

Failure Atomicity and Structural State Consistency

A failed method invocation should leave the object in the state it occupied prior to the call.³

This is known as failure atomicity.³

To preserve failure atomicity, developers should validate parameters *before* performing any state-changing operations.³ Other effective approaches include using immutable objects (which possess static state stability) or writing recovery blocks that roll back state changes if an error occurs.³

Java

```
// Non-Compliant: State mutated before parameter validation, violating failure atomicity
public void processBatch(java.util.List<String> items)
    this.processedCount += items.size() // State mutated first
    if (items.contains(null))
        throw new IllegalArgumentException("Batch contains invalid null entries") // State is now
corrupted
```

Java

```
// Compliant: Validation occurs before state mutation
public void processBatch(java.util.List<String> items)
    if (items == null || items.contains(null))
        throw new IllegalArgumentException("Batch contains invalid null entries") // Intercepts
errors first
    this.processedCount += items.size() // State mutated only after validation succeeds
```

AI reviewers can also evaluate when error codes are used within exceptions.²¹ Returning a static enum error code can be useful when mapping back-end failures to front-end clients or REST APIs.²¹ However, using explicit subclassed exceptions is preferred inside the domain layer, as it allows clean polymorphic catching.²¹

```
Java
// Non-Compliant: Error code used inside domain logic
try
    paymentService.charge()
catch (PaymentException e)
    if (e.getErrorCode() == ErrorType.INSUFFICIENT_FUNDS) // Tightly coupled conditional
        block
            remedyBalance()
    }
```

```
Java
// Compliant: Clean polymorphic catching via subclassed exceptions
try
    paymentService.charge()
catch (InsufficientFundsException e)
    remedyBalance() // Polymorphically intercepted
catch (PaymentException e)
    logGeneralFailure(e)
}
```

Concurrency, Memory Barriers, and Thread Coordination

To review multi-threaded code, an AI reviewer must understand the Java Memory Model (JMM), thread synchronization mechanics, and lock safety guidelines.

Memory Visibility and Volatile Flags under the Java Memory Model

Under the JMM, threads may cache variables locally within CPU registers or L1/L2 caches to optimize execution. Without explicit memory barriers, writes to a variable by one thread may not be visible to reader threads.

The volatile keyword establishes a happens-before relationship, forcing all writes directly to main memory and ensuring subsequent reads are pulled directly from main memory.³

```
Java
// Non-Compliant: Thread caching can prevent visibility of state changes, causing infinite loops
public class WorkerTask implements Runnable {
    private boolean active = true;

    public void run() {
        while (active) {
            performWork();
        }
    }

    public void stop() {
        this.active = false; // Writes might never be observed by the loop thread
    }

    private void performWork() {
    }
}
```

```
Java
// Compliant: Volatile modifier guarantees memory visibility
public class WorkerTask implements Runnable {
    private volatile boolean active = true;

    public void run() {
        while (active) {
            performWork();
        }
    }
}
```

```

    public void stop()
    {
        this.active = false // Synchronously visible across distinct execution contexts
    }

    private void performWork()
    {

```

Synchronization Boundaries, Lock Contention, and Monitor Lockouts

Synchronization locks must be restricted to minimal critical sections to limit lock contention.²⁰ A critical concurrency error is synchronizing on string literal constants (such as `synchronized("LOCK")`). Because string literals are pooled across the JVM, synchronizing on a literal string can block thread execution across unrelated classes and libraries.

```

Java
// Non-Compliant: Locking on a shared literal string causes global lock contention
public class OrderProcessor
{
    private static final String LOCK = "ORDER_LOCK"

    public void processOrder()
    {
        synchronized (LOCK) // Collides with other classes synchronizing on the same literal
        {
            executeProcessing()
        }
    }

    private void executeProcessing()
    {

```

```

Java
// Compliant: Locking on a private final object instance
public class OrderProcessor
{
    private final Object processLock = new Object() // Fully encapsulated, isolated lock

```

```

public void processOrder()
    synchronized (processLock)
        executeProcessing()
}
private void executeProcessing()

```

In addition to lock contention, AI code reviews must identify nested monitor lockouts, busy waiting patterns (which waste CPU cycles), and forgotten synchronization blocks.²⁰ The use of wait/notify mechanisms should be evaluated and, where appropriate, refactored to use higher-level concurrency utilities such as CountdownLatch, CyclicBarrier, or Condition.²⁰

```

Java
// Non-Compliant: Busy waiting loop consumes excessive CPU cycles
public class LatencyBuffer
    private boolean completed = false

    public void awaitCompletion()
        while (!completed)
            // Wastes CPU cycles spinning continuous validation loops
}

```

```

Java
// Compliant: Concurrency latch suspends threads efficiently
public class LatencyBuffer
    private final java.util.concurrent.CountDownLatch latch = new
        java.util.concurrent.CountDownLatch(1)

```



```

    public void awaitCompletion() throws InterruptedException
    {
        latch.await(); // Suspends thread execution efficiently, releasing CPU resources
    }

    public void complete()
    {
        latch.countDown(); // Releases waiting threads
    }
}

```

Request-State Bleed in Singleton Components

Many runtime systems manage service, repository, and controller beans as shared Singletons.²² Defining non-final fields inside these components to manage transient request-specific state creates direct state corruption and security leaks when accessed concurrently by multiple thread pools.²²

```

Java
// Non-Compliant: Controller field modified inside request threads causes cross-user state bleed
public class OrderController
{
    private Order currentRequestOrder; // Vuln: Shared field overwritten concurrently

    public void processOrder(Order order)
    {
        this.currentRequestOrder = order;
        executeValidation();
    }

    private void executeValidation()
    {
        if (currentRequestOrder.getAmount() <= 0)
        {
            throw new IllegalArgumentException();
        }
    }
}

```

Java

// Compliant: Keep components stateless; restrict parameters to the execution stack

```
public class OrderController
```

```
    public void processOrder(final Order order)
```

```
    {
        executeValidation(order); // Confined cleanly to thread-local stack frames
    }

```

```
    private void executeValidation(final Order order)
```

```
    {
        if (order.getAmount() <= 0
```

```
        )
            throw new IllegalArgumentException();
    }

```

Thread-Safe Violations and ThreadPool Configuration

Some commonly used JDK classes (such as SimpleDateFormat and Calendar) are not thread-safe.²³ Storing instances of these classes in static variables can lead to data corruption in multi-threaded environments.²³ AI reviewers should recommend replacing these outdated classes with modern, immutable date-time classes from the java.time package, which are inherently thread-safe.²³

Java

// Non-Compliant: SimpleDateFormat is not thread-safe, leading to date corruption

```
public class LogFormatter
```

```
    private static final java.text.SimpleDateFormat FORMATTER = new
```

```
    java.text.SimpleDateFormat("yyyy-MM-dd")
```

```
    public static String formatDate(java.util.Date date)
```

```
    {
        return FORMATTER.format(date); // Multiple threads accessing this format call will corrupt
    }

```

internal state

Java

// Compliant: InstantFormatter utilizes thread-safe, immutable DateTimeFormatter

```
public class LogFormatter
```

```
    private static final java.time.format.DateTimeFormatter FORMATTER =
```

```
        java.time.format.DateTimeFormatter.ISO_LOCAL_DATE;
```

```
    public static String formatDate(java.time.LocalDate date)
```

```
    {
        return FORMATTER.format(date); // Thread-safe execution under concurrent loads
    }
}
```

In concurrent environments, managing singletons also requires care.²⁰ Double-checked locking singletons are highly fragile and can cause concurrency bugs if the singleton field is not marked volatile.²⁰ A robust, thread-safe alternative is the Initialization-on-demand holder idiom, which relies on the JVM's class-loading guarantees to ensure safe, lazy initialization.²⁰

Java

// Non-Compliant: Double-checked locking singleton is vulnerable if field is not volatile

```
public class CacheLoader
```

```
    private static CacheLoader instance;
```

```
    private CacheLoader()
```

```
    public static CacheLoader getInstance()
```

```
    {
        if (instance == null
```

```
        {
            synchronized (CacheLoader.class) {
```

```
                if (instance == null
```

```
                {
                    instance = new CacheLoader(); // Vulnerable to out-of-order writes
                }
            }
        }
    }
}
```

```
return instance;
```

```
Java
```

```
// Compliant: Initialization-on-demand holder idiom guarantees thread-safe, lazy loading
```

```
public class CacheLoader
```

```
    private CacheLoader()
```

```
    private static class Holder
```

```
    {
        private static final CacheLoader INSTANCE = new CacheLoader(); // Thread-safe JVM
```

```
    }
}
```

```
public static CacheLoader getInstance()
```

```
    {
        return Holder.INSTANCE;
    }
}
```

Enterprise Security and OWASP Top 10 Defect Signatures

Evaluating code safety is a crucial task for an automated reviewer.⁹ This section establishes specific code defect signatures mapping to OWASP security parameters, ensuring the model can detect and rewrite vulnerable patterns.

SQL Injection vulnerabilities

Constructing dynamic database queries via string concatenation represents a severe security risk, allowing command modification and host takeover. PreparedStatements are necessary to isolate code execution parameters from dynamic data.

Java

// Non-Compliant: SQL queries constructed through string concatenation are highly vulnerable

```
public void updateRole(String username, String role) throws SQLException
```

```
    Connection conn = dataSource.getConnection();
```

```
    Statement stmt = conn.createStatement();
```

```
    String query = "UPDATE users SET role = '" + role + "' WHERE username = '" + username + "'";
```

```
    stmt.executeUpdate(query); // Injecting "admin' OR '1'=1" allows unauthorized privilege
```

```
    escalation
```

```
}
```

Java

// Compliant: Use parameter binding with PreparedStatements to enforce data separation

```
public void updateRole(String username, String role) throws SQLException
```

```
    String sql = "UPDATE users SET role =? WHERE username =?";
```

```
    try (Connection conn = dataSource.getConnection())
```

```
    {
```

```
        PreparedStatement pstmt = conn.prepareStatement(sql);
```

```
        pstmt.setString(1, role);
```

```
        pstmt.setString(2, username);
```

```
        pstmt.executeUpdate(); // Secure data execution isolated from code
```

```
    }
```

Unvalidated Redirects and SSRF (Server-Side Request Forgery)

Extracting a dynamic redirect URL parameter directly from an HTTP request and redirecting users without validation represents an Open Redirect vulnerability. This can be abused in phishing attacks to route users to untrusted registration systems.

Java

// Non-Compliant: Open redirection permits unvalidated user routing

```
public class RedirectServlet extends HttpServlet
```

```
    {
```

```
        protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException
```

```
        {
```

```
String destination = req.getParameter("url")
resp.sendRedirect(destination) // Vuln: Redirects directly to external malicious domain

```

Java

// Compliant: Enforce redirection boundaries using explicit domain whitelists

```
public class RedirectServlet extends HttpServlet
```

```
    private static final Set<String> ALLOWED_DOMAINS = Set.of("enterprise.com",
"api.enterprise.com")
```

```
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException
```

```
    {
        String destination = req.getParameter("url")
```

```
        if (!isValidDomain(destination))
```

```
        {
            resp.sendRedirect(destination)
```

```
        }
        else
```

```
        {
            resp.sendError(HttpServletResponse.SC_BAD_REQUEST, "Unauthorized destination
redirect.")
        }
    }

```

```
    private boolean isValidDomain(String url)
```

```
    {
        try
```

```
        {
            java.net.URI uri = new java.net.URI(url)
```

```
            String host = uri.getHost()
```

```
            return host != null && ALLOWED_DOMAINS.contains(host)
```

```
        }
        catch (Exception e)
```

```
        {
            return false
        }
    }

```

Insecure Directory Permissions & Path Traversal

When uploading and managing file paths, using loose access permissions or neglecting to validate filenames against path traversal sequences (../) exposes the host system to unauthorized file manipulation.

Java

// Non-Compliant: Creating paths with wildcard write permissions and unvalidated names

```
public void processUpload(HttpServletRequest req, Part filePart) throws IOException
```

```
    Path uploadDir = Path.of("/var/www/uploads")
```

```
    Files.createDirectories(uploadDir)
```

```
    // Vuln: Grants full execution rights to any system user
```

```
    Files.setPosixFilePermissions(uploadDir, PosixFilePermissions.fromString("rwxrwxrwx"))
```

```
    String filename = filePart.getSubmittedFileName() // Path traversal danger if filename is
```

```
    "../../../etc/passwd"
```

```
    Path targetFile = uploadDir.resolve(filename)
```

```
    Files.copy(filePart.getInputStream(), targetFile)
```

```
}
```

Java

// Compliant: Restrict file permissions and validate names against path containment

```
public void processUpload(HttpServletRequest req, Part filePart) throws IOException
```

```
    Path uploadDir = Path.of("/var/www/uploads")
```

```
    Files.createDirectories(uploadDir)
```

```
    // Secure: Access restricted to application owner group
```

```
    Files.setPosixFilePermissions(uploadDir, PosixFilePermissions.fromString("rwxr-x---"))
```

```
    String filename = filePart.getSubmittedFileName()
```

```
    if (filename == null || filename.contains(".."))
```

```
        throw new SecurityException("Potential path traversal attempt detected.")
```

```
}
```

```
    Path targetFile = uploadDir.resolve(filename).normalize()
```

```
    if (!targetFile.startsWith(uploadDir))
```

```
        throw new SecurityException("Target file resolves outside of authorized directory boundaries.")
```



AI Training Datasets, Fine-Tuning Benchmarks, and Evaluation Frameworks

To build an AI code reviewer, training datasets must provide high-quality code representations and clear feedback loops.⁷ These datasets need to cover standard code repair tasks as well as complex, multi-file structural modifications.¹⁰

Multi-Task Code Intelligence and Dataset Scale

To train an AI code reviewer, the dataset must include a mix of large-scale code corpora and highly curated benchmarks.⁷ These sources can be used for pre-training, Supervised Fine-Tuning (SFT), and Reinforcement Learning (RL).⁹

- **Java-Code-Large**²⁴: A massive corpus containing over 10 million rows of Java code.²⁴ It is ideal for pre-training models on raw syntax, package architectures, and standard API usages.²⁴
- **github-codereview**²⁷: A dataset focused on pull request commits and human review comments.²⁷ It helps train models to understand developer discussions and suggest context-aware changes.²⁷
- **Code-Feedback**²⁸: A dataset containing over 60,000 multi-turn programming instructions.²⁸ It is highly useful for SFT, teaching models how to handle refactoring requests and optimize runtime space and execution parameters.²⁸

Program Repair and Defect Detection Frameworks

To evaluate defect detection and automated repair capabilities, models should be tested against established industry benchmarks.¹⁰

- **CodeXGLUE**¹⁰: A comprehensive multi-task benchmark containing 14 datasets across 10 programming language tasks.³⁰ It includes clone detection, defect prediction, token completion, and automated repair (via the Bugs2Fix dataset).¹
- **Defects4J**²⁵: A manually curated database of over 350 real-world bugs drawn from popular open-source Java projects (such as Closure Compiler, Commons Math, and Joda-Time).²⁵ Each bug in Defects4J is paired with a triggering test suite and explicit metadata, making it a valuable benchmark for testing an AI's ability to locate and fix complex bugs.²⁵

Refactoring Quality and Cross-File Structural Reasonings

Traditional code benchmarks evaluate code quality using simple functional correctness metrics, such as the Test Pass Rate (TPR).³³ However, evaluating an AI reviewer's ability to

maintain long-term code quality requires assessing structural improvements beyond simple functional tests.³³

SmellBench is an extensible benchmark designed to evaluate code agents on refactoring tasks by injecting complex code smells into clean codebases.³⁴ SmellBench evaluates code refactoring across seven representative code smell types: Feature Envy, God Classes, Data Clumps, Shotgun Surgery, Dead Code Elimination, Interface Segregation, and Deep Inlining.³⁴

Smell Type	Architectural Definition	Complexity Vector
Feature Envy ³⁴	A method is more interested in the data of another class than its own ³⁴	Requires localized method extraction and relocation ³⁴
God Classes ³⁴	A single class aggregates excessive, unrelated responsibilities ³⁴	Requires high-level decomposition and subclass delegations ³⁵
Data Clumps ³⁴	A set of identical parameters frequently appears together in methods ³⁴	Requires parameter consolidation into structured domain objects ³⁴
Shotgun Surgery ³⁴	Making a change requires modifying many small code locations across multiple classes ³⁴	High complexity; requires cross-file coordination across many files ³⁴
Dead Code Elimination ³⁴	Unused variables, methods, or unreachable classes persist in the codebase ³⁴	Requires dependency tree evaluations ³⁴
Interface Segregation ³⁴	Large, fat interfaces force implementing classes to depend on unused methods ³⁴	Requires interface decomposition into narrow, single-purpose structures ³⁴
Deep Inlining ³⁴	Excessive nested method call chains degrade legibility ³⁴	Requires targeted, semantic inlining operations ³⁴

Evaluation results on SmellBench highlight the limitations of modern AI models.³³ While

high-tier models (like Qwen Code and Claude 3.5 Sonnet) pass over 80% of functional tests (high TPR), they achieve only a 50.34% score on actual smell elimination.³³

This performance gap occurs because models struggle with cross-file understanding.³⁴ For example, refactoring "Shotgun Surgery" requires making coordinated changes across an average of 5.7 files, but models typically edit only 2.7 files.³⁴

Therefore, a robust training dataset for an AI code reviewer must look beyond simple functional tests.³³ It must incorporate metrics like Localization Accuracy (LA) and structural quality assessments (using LLM-as-Judge frameworks) to ensure the AI learns to write compilable, secure, and highly maintainable code.³³

Static Code Analysis Rules and AST Specifications

To train an AI code reviewer, the dataset must also include formal validation rules that map syntactic patterns directly to abstract syntax trees (ASTs).³⁶ The following schemas illustrate how specific PMD XPath rules can be used to identify and enforce structural best practices in Java.³⁷

1. Enforcing Narrowest Variable Scope

This rule flags variables declared with an unnecessarily broad scope, encouraging developers to keep variable declarations close to their first point of use.³⁷

```
XML
<rule name="AvoidUnnecessaryWideScope" language="java" message="Minimize the scope of
variables to prevent state leakage." class="net.sourceforge.pmd.lang.rule.XPathRule">
  <description> Ensure local variables are declared close to their point of use rather than at
the top of method blocks. </description>
  <priority> </priority>
  <properties>
    <property name="xpath">
      <value>
        //
      </value>
    </property>
  </properties>
</rule>
```

2. Eliminating Unnecessary Protected Visibility

This rule flags protected members declared in classes that are marked as final and do not inherit from other structures.³⁷ Since a final class cannot be subclassed, protected visibility is redundant and should be changed to private or package-private.³⁷

```
XML
<rule name="AvoidProtectedMethodInFinalClassNotExtending" language="java" message="Final
classes cannot be subclassed; protected visibility is redundant."
class="net.sourceforge.pmd.lang.rule.XPathRule">
  <description> flags protected methods inside final classes that do not inherit from parent
structures, as subclassing is blocked. </description>
  <priority> </priority>
  <properties>
    <property name="xpath">
      <value>
        //ClassOrInterfaceBody/ClassOrInterfaceBodyDeclaration
        /MethodDeclaration[@Protected = true() and @Name != 'finalize']]
      </value>
    </property>
  </properties>
</rule>
```

3. Restricting Non-Standard Naming Conventions

This rule flags the use of special characters (like \$) in class, variable, or method identifiers, ensuring the codebase adheres to standard naming conventions.³⁷

```
XML
<rule name="AvoidDollarSigns" language="java" message="Do not use dollar signs in variable,
method, or class names."
class="net.sourceforge.pmd.lang.java.rule.codestyle.AvoidDollarSignsRule">
  <description> dollar signs are reserved for nested classes and compiler operations; using
them in standard identifiers is discouraged. </description>
```

```
<priority> </priority>
</rule>
```

Master Dataset Schema Specification (JSONL Format)

The master dataset should be structured as a single-line JSON Lines (JSONL) file to support efficient parsing during training.²⁶ The following examples show how complex code reviews can be formatted into tokenized instruction-response pairs:

JSON

```
{
  "instance_id" "MSR_JAVA_2026_09841"
  "metadata"
    "problem_domain" "Memory Management & Concurrency"
    "difficulty" "Hard"
    "targeted_smell" "ThreadLocal Pool Leak"
    "pmd_rule_reference" "category/java/errorprone.xml/UnusedNullAssignment"
  "context"
    "repository" "apache/commons-pool"
    "jdk_version" "Java 17"
    "problem_statement" "An accumulation of classloaders and metadata causes
OutOfMemoryError in hot redeloys due to un-evicted ThreadLocal keys on pooled worker
threads."
    "original_code" "package org.apache.commons.pool;\n\nimport java.util.UUID;\n\npublic class
TransactionContext {\n  private static final ThreadLocal<String> transactionId = new
ThreadLocal<>();\n\n  public static void beginTransaction() {\n
transactionId.set(UUID.randomUUID().toString());\n  }\n\n  public static String getTransactionId()
{\n    return transactionId.get();\n  }\n}"
    "refactored_code" "package org.apache.commons.pool;\n\nimport java.util.UUID;\n\npublic class
TransactionContext {\n  private static final ThreadLocal<String> transactionId = new
ThreadLocal<>();\n\n  public static void beginTransaction() {\n
transactionId.set(UUID.randomUUID().toString());\n  }\n\n  public static String getTransactionId()
{\n    return transactionId.get();\n  }\n\n  public static void clearTransaction() {\n
transactionId.remove();\n  }\n}"
    "structured_review"
    "severity" "Critical"
```

```
"line_range" 1 2
"diagnostic" "ThreadLocal variable transactionId is populated inside worker threads but never
cleared using the.remove() method."
"remediation_justification" "In managed servlet environments, worker threads persist across
hot redeployments. Failure to evict ThreadLocal data creates reference chains that prevent the
ClassLoader from being garbage collected, resulting in Metaspace and Heap depletion."

```

JSON

```
"instance_id" "MSR_JAVA_2026_11304"
"metadata"
"problem_domain" "Design Patterns & Creational Mechanics"
"difficulty" "Medium"
"targeted_smell" "Beans Thread Safety Violation"
"pmd_rule_reference" "category/java/errorprone.xml/AvoidInstanceVariablesInSingletons"
"context"
"repository" "spring-projects/spring-framework"
"jdk_version" "Java 21"
"problem_statement" "The OrderController class, configured as a Singleton bean, stores
transient request parameters in its fields, leading to data leaks across concurrent requests."
"original_code" "package com.enterprise.controller;\n\nimport
com.enterprise.model.Order;\nimport com.enterprise.service.OrderService;\n\npublic class
OrderController {\n    private final OrderService orderService;\n    private Order
currentRequestOrder;\n\n    public OrderController(OrderService orderService) {\n
this.orderService = orderService;\n    }\n\n    public void processOrder(Order order) {\n
this.currentRequestOrder = order;\n    orderService.validate(currentRequestOrder);\n
orderService.persist(currentRequestOrder);\n    }\n}"
"refactored_code" "package com.enterprise.controller;\n\nimport
com.enterprise.model.Order;\nimport com.enterprise.service.OrderService;\n\npublic class
OrderController {\n    private final OrderService orderService;\n\n    public
OrderController(OrderService orderService) {\n    this.orderService = orderService;\n    }\n\n
public void processOrder(Order order) {\n    orderService.validate(order);\n
orderService.persist(order);\n    }\n}"
```

```
"structured_review"  
"  
"severity" "Critical"  
"line_range" 3 4  
"diagnostic" "The singleton bean OrderController contains a non-final instance variable  
currentRequestOrder used to store transient request state."  
"remediation_justification" "Since controllers are singletons, concurrent request threads share  
this class instance. Storing request data in instance fields causes data leaks, as different threads will  
overwrite this shared field simultaneously."  
"  
"  
"
```

JSON

```
"instance_id" "MSR_JAVA_2026_14502"  
"metadata"  
"problem_domain" "Null Safety & Optional Mechanics"  
"difficulty" "Medium"  
"targeted_smell" "Eager Evaluation Resource Waste"  
"pmd_rule_reference" "category/java/bestpractices.xml/UseOrElseGet"  
"  
"context"  
"repository" "reactive-web/router"  
"jdk_version" "Java 21"  
"problem_statement" "The API endpoint routing fallback triggers expensive database lookups  
eagerly within orElse, degrading performance even on cache hits."  
"  
"original_code" "package com.reactive.router;\n\nimport com.reactive.model.Route;\nimport  
java.util.Optional;\n\npublic class RouteResolver {\n    private final RouteCache cache;\n    private  
final RouteRepository db;\n\n    public RouteResolver(RouteCache cache, RouteRepository db) {\nthis.cache = cache;\n        this.db = db;\n    }\n\n    public Route getActiveRoute(String routeld) {\nOptional<Route> cached = cache.get(routeld);\n        return  
cached.orElse(db.fetchFromPrimaryStorage(routeld));\n    }\n}"  
"refactored_code" "package com.reactive.router;\n\nimport com.reactive.model.Route;\nimport  
java.util.Optional;\n\npublic class RouteResolver {\n    private final RouteCache cache;\n    private  
final RouteRepository db;\n\n    public RouteResolver(RouteCache cache, RouteRepository db) {\nthis.cache = cache;\n        this.db = db;\n    }\n\n    public Route getActiveRoute(String routeld) {\n
```

```
Optional<Route> cached = cache.get(routeId);\n    return cached.orElseGet(() ->\n        db.fetchFromPrimaryStorage(routeId));\n    }\n}"
"structured_review"
"severity" "Major"
"line_range" 17 17
"diagnostic" "Using orElse with an expensive method call causes the method to execute eagerly,\neven when the Optional contains a valid value."
"remediation_justification" "Refactoring to orElseGet with a lambda supplier defers execution of\nfetchFromPrimaryStorage, which eliminates unnecessary database connections and CPU cycles\nduring cache hits."

```

JSON

```
"instance_id" "MSR_JAVA_2026_18903"
"metadata"
"problem_domain" "Application Security"
"difficulty" "Hard"
"targeted_smell" "SQL Injection Pattern"
"pmd_rule_reference" "category/java/security.xml/AvoidUsingUnsafeSql"
"context"
"repository" "spring-projects/spring-security"
"jdk_version" "Java 17"
"problem_statement" "An unsafe SQL statement constructed with user input concatenation\ncreates an injection risk within JDBC data repositories."
"original_code" "package com.enterprise.repository;\n\nimport java.sql.*;\nimport\njavax.sql.DataSource;\n\npublic class AccountRepository {\n    private final DataSource ds;\n\n    public AccountRepository(DataSource ds) {\n        this.ds = ds;\n    }\n\n    public void\ndeactivateUser(String username) throws SQLException {\n        try (Connection conn =\nds.getConnection();\n            Statement stmt = conn.createStatement()) {\n            String sql =\n                \"UPDATE accounts SET status = 'INACTIVE' WHERE username = '\" + username + '\"\";\n            stmt.executeUpdate(sql);\n        }\n    }\n}"
"refactored_code" "package com.enterprise.repository;\n\nimport java.sql.*;\nimport"

```

```

javax.sql.DataSource;\n\npublic class AccountRepository {\n    private final DataSource ds;\n\n    public AccountRepository(DataSource ds) {\n        this.ds = ds;\n    }\n\n    public void
deactivateUser(String username) throws SQLException {\n        String sql = \"UPDATE accounts SET
status = 'INACTIVE' WHERE username = ?\";\n        try (Connection conn = ds.getConnection();\n        PreparedStatement pstmt = conn.prepareStatement(sql)) {\n            pstmt.setString(1, username);\n
pstmt.executeUpdate();\n        }\n    }\n}

```

"structured_review"

"severity" "Critical"

"line_range" 5 4

"diagnostic" "SQL statement constructed by concatenating untrusted parameters directly into the query string."

"remediation_justification" "String concatenation allows attackers to manipulate the SQL statement structure (SQL Injection). Using parameter binders in a PreparedStatement isolates SQL logic from execution parameters, neutralising payload executions."

Works cited

1. Home · microsoft/CodeXGLUE Wiki · GitHub, accessed on June 12, 2026, <https://github.com/microsoft/CodeXGLUE/wiki>
2. Make your Java code smell nice and fresh - freeCodeCamp, accessed on June 12, 2026, <https://www.freecodecamp.org/news/do-not-allow-bad-smells-in-your-java-code-4e8ad244393/>
3. learning-notes/books/effective-java.md at master · keyvanakbary ..., accessed on June 12, 2026, <https://github.com/keyvanakbary/learning-notes/blob/master/books/effective-java.md>
4. ThreadLocal & Memory Leak - java - Stack Overflow, accessed on June 12, 2026, <https://stackoverflow.com/questions/17968803/threadlocal-memory-leak>
5. Java Memory Leaks: What Causes Them and How to Avoid Them - DEV Community, accessed on June 12, 2026, <https://dev.to/skilledcoder/java-memory-leaks-what-causes-them-and-how-to-avoid-them-469b>
6. Understanding Memory Leaks in Java | Baeldung, accessed on June 12, 2026, <https://www.baeldung.com/java-memory-leaks>
7. GitHub - in28minutes/java-best-practices: Best practices in Coding, Designing and Architecting Java Applications, accessed on June 12, 2026, <https://github.com/in28minutes/java-best-practices>
8. [2603.00821] Taking a Closer Look at Warnings Generated by PMD and SonarQube, their Rules and Compliance to Established Coding Standards - arXiv, accessed on June 12, 2026, <https://arxiv.org/abs/2603.00821>

9. Source code review: A comprehensive guide to secure development - Sonar, accessed on June 12, 2026, <https://www.sonarsource.com/resources/library/source-code-review/>
10. Datasets for code · Issue #28 · huggingface/open-r1 - GitHub, accessed on June 12, 2026, <https://github.com/huggingface/open-r1/issues/28>
11. Java Anti-Patterns, accessed on June 12, 2026, <https://www.odi.ch/prog/design/newbies.php>
12. The 3 Rules of Clean Java Code (That Most Devs Break) - YouTube, accessed on June 12, 2026, <https://www.youtube.com/watch?v=mrB399ajzs>
13. Bad Java code in 7 easy steps - Andreas Maier - Medium, accessed on June 12, 2026, <https://astromaier.medium.com/bad-java-code-in-7-easy-steps-792eee03855b>
14. Google Java Style Guide, accessed on June 12, 2026, <https://google.github.io/styleguide/javaguide.html>
15. 15-121 Java Style Guide - Carnegie Mellon University in Qatar, accessed on June 12, 2026, <https://web2.qatar.cmu.edu/cs/15121/resources/styleguide/>
16. kodustech/awesome-code-review-tools - GitHub, accessed on June 12, 2026, <https://github.com/kodustech/awesome-code-review-tools>
17. The Unseen Memory Leak: How ThreadLocal Variables Can Bring Down Your Application, accessed on June 12, 2026, <https://blog.heaphero.io/threadlocal-memory-leak/>
18. Memory leak as a result of no cleanup for ThreadLocal in CodedOutputStream?? · Issue #7083 · open-telemetry/opentelemetry-java - GitHub, accessed on June 12, 2026, <https://github.com/open-telemetry/opentelemetry-java/issues/7083>
19. Exception Handling in Java - Baeldung, accessed on June 12, 2026, <https://www.baeldung.com/java-exceptions>
20. Java concurrency patterns for educational purposes - GitHub, accessed on June 12, 2026, <https://github.com/alxkm/java-concurrency-patterns>
21. Java Exception Error Enumerations Anti-pattern - Software Engineering Stack Exchange, accessed on June 12, 2026, <https://softwareengineering.stackexchange.com/questions/444232/java-exception-error-enumerations-anti-pattern>
22. Two Common Concurrency Bugs - Java Code Geeks, accessed on June 12, 2026, <https://www.javacodegeeks.com/2020/01/two-common-concurrency-bugs.html>
23. What is the most frequent concurrency issue you've encountered in Java? - Stack Overflow, accessed on June 12, 2026, <https://stackoverflow.com/questions/461896/what-is-the-most-frequent-concurrency-issue-youve-encountered-in-java>
24. ajibawa-2023/Java-Code-Large · Datasets at Hugging Face, accessed on June 12, 2026, <https://huggingface.co/datasets/ajibawa-2023/Java-Code-Large>
25. Defects4J V1.2 Benchmark - Emergent Mind, accessed on June 12, 2026, <https://www.emergentmind.com/topics/defects4j-v1-2>
26. github/CodeSearchNet: Datasets, tools, and benchmarks for representation learning of code., accessed on June 12, 2026, <https://github.com/github/CodeSearchNet>

27. ronantakizawa/github-codereview · Datasets at Hugging Face, accessed on June 12, 2026, <https://huggingface.co/datasets/ronantakizawa/github-codereview>
28. HuggingFaceH4/Code-Feedback · Datasets at Hugging Face, accessed on June 12, 2026, <https://huggingface.co/datasets/HuggingFaceH4/Code-Feedback>
29. CodeXGLUE, accessed on June 12, 2026, <https://microsoft.github.io/CodeXGLUE/>
30. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation - arXiv, accessed on June 12, 2026, <https://arxiv.org/pdf/2102.04664>
31. anti-pattern · GitHub Topics, accessed on June 12, 2026, <https://github.com/topics/anti-pattern?o=asc&s=forks>
32. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs - University of Washington, accessed on June 12, 2026, https://homes.cs.washington.edu/~rjust/publ/defects4j_issta_2014.pdf
33. SmellBench: Towards Fine-Grained Evaluation of Code Agents on Refactoring Tasks - arXiv, accessed on June 12, 2026, <https://arxiv.org/pdf/2606.05574>
34. [Literature Review] SmellBench: Towards Fine-Grained Evaluation of Code Agents on Refactoring Tasks - Moonlight, accessed on June 12, 2026, <https://www.themoonlight.io/review/smellbench-towards-fine-grained-evaluation-of-code-agents-on-refactoring-tasks>
35. SmellBench: Towards Fine-Grained Evaluation of Code Agents on Refactoring Tasks - arXiv, accessed on June 12, 2026, <https://arxiv.org/abs/2606.05574>
36. Writing Custom Pmd Rules, accessed on June 12, 2026, <https://sites.google.com/view/writing-custom-pmd-rules-52/>
37. Code Style | PMD Source Code Analyzer, accessed on June 12, 2026, https://pmd.github.io/pmd/pmd_rules_java_codestyle.html
38. Code Style | PMD Source Code Analyzer, accessed on June 12, 2026, https://docs.pmd-code.org/pmd-doc-6.55.0/pmd_rules_java_codestyle.html